Prompt Router for Multi-LLM Systems

Jayashree Bagade
Department of Information Technology
Vishwakarma Institue of Technology
Pune, Maharashtra
jayashree.bagade@vit.edu

Anish Katariya
Department of Information Technology
Vishwakarma Institue of Technology
Pune, Maharashtra
anish.katariya23@vit.edu

Ninad Kale
Department of Information Technology
Vishwakarma Institue of Technology
Pune, Maharashtra
ninad.kale23@vit.edu

Amey Kharade
Department of Information Technology
Vishwakarma Institue of Technology
Pune, Maharashtra
amey.kharade231@vit.edu

Manasi Phand
Department of Information Technology
Vishwakarma Institue of Technology
Pune, Maharashtra
manasi.phand23@vit.edu

Abstract*—Large Language Models (LLMs) play an indispensable role in today's applications, from chatbots to virtual assistants to decision-making systems. As more and more different LLMs emerge, however, there appears a problem of effective management of different LLMs with distinct costs, latencies, and reasoning capabilities. Current systems mostly employ a static approach by selecting only one LLM and sending all queries to it. Such an approach is far from being optimal since it results in inefficient use of resources, higher operational costs, and poor performance. In order to solve this problem, the current paper proposes a Prompt Router for Multi-LLM Systems – a learning framework for dynamic and effective routing. The router learns query embeddings of size 1536 and applies a neural model based on matrix factorization to learn query compatibility with LLMs. The training dataset consists of over 29,000 samples of comparison between pairs of LLMs and employs binary cross-entropy loss. After training, the router predicts probabilities of one model outperforming another model and sends queries to either light-weight or high-performing models based on this prediction. Experimental evaluation on MT-Bench demonstrates that the proposed router achieves performance comparable to static baselines while significantly improving efficiency. Specifically, the system reduces average inference cost by approximately 25–30% and lowers latency by around 20%, while maintaining competitive response quality within 2–3% of strong-model baselines. These results highlight the potential of learning-based routing for balancing cost, latency, and performance in multi-LLM systems.

Keywords— LLM Routing, Multi-LLM Systems, Prompt Routing, Matrix Factorization, Query Embeddings, Model Selection.

I. INTRODUCTION

The emergence of Large Language Models (LLM) has marked the transformation of artificial intelligence because the machines are now able to undertake complicated language comprehension and language creation tasks seamlessly. Modern applications, including chatbots, intelligent teaching applications, intelligent customer service, law or medicine helpers, and corporate knowledge bases among others rely on LLMs to power their reasoning engines. They demonstrate high-level performance in tasks, such as question answering, summarizing, translation, code creation, logical thinking, and decision making. Nevertheless, fast progress in LLM technology has led to the emergence of numerous models with varying computational complexity, cost models, inference latency, context window size, and depth of reasoning. This created an intricate set of performance trade-offs for any business trying to utilize AI solutions. Usually, high-performance LLMs are characterized by superior reasoning skills, better contextual comprehension, and higher accuracy on complicated tasks. At the same time, such LLMs feature greater computational complexity, higher API costs, and more latency. By contrast, lightweight models are characterized by low latency and low costs; yet, they demonstrate poor results when undertaking complicated analytical tasks. The difference is essential, especially if an organization is working in a production environment, which needs to process thousands or millions of queries daily. A system that randomly directs all queries to a powerful and expensive model may be highly accurate but economically unsustainable. Similarly, directing all queries to lightweight models may be economically sound but may result in decreased user satisfaction due to decreased response quality on complex tasks. Thus, efficient and intelligent model usage has become a pressing challenge in large-scale AI deployments.

Despite the existence of many models, most current models are still based on the static single-model paradigm: every query goes to a default LLM. This one-size-fits-all approach fails to take advantage of the strengths of different models and is inefficient. Simple questions requiring little reasoning spend expensive compute, and complex, time-sensitive, or high-priority queries may not be processed optimally. With the growing number of models available and dynamic pricing, more sophisticated and intelligent routing is needed. Prompt routing, the automatic selection of the best LLM for a given query, has been shown to be a promising direction in multi-LLM systems. Recent research has investigated different routing approaches, ranging from cost-conscious cascades and preference-driven routing to reinforcement learning-based policies and confidence-driven routing. These studies demonstrate that effective routing can reduce costs or enhance quality, but most routing architectures are designed to optimize a single objective. In practical applications, routing needs to consider several goals at once – cost, latency, and quality. Optimal routing using single-objective optimization may become inefficient since concentrating only on reducing the costs may result in higher error rate, and vice versa. Lack of multi-objective optimization platforms will make it impossible to achieve scaling up.

Thus, this paper proposes a Prompt Router for Multi-LLM System, which can be implemented as a machine learning-based middleware. The middleware analyses input prompts and extracts such features as their size, complexity of context provided, type of task performed, and inference of required reasoning. Then using performance data of multiple models, a supervised classifier is constructed and taught to predict which of the models should be applied. As opposed to conventional or rule-based routing, this approach ensures that the model selection process takes into account all three parameters discussed above. Using the proposed Prompt Router, one is able to make more informed decisions on model selection,

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

optimize their usage from both performance and economic perspectives, and ensure scalability to practical multi-LLM systems. Therefore, it is possible to conclude that machine learning fundamentals like feature engineering, supervised classification, and performance-based labelling allow developing solutions for increasingly complex problems related to intelligent routing.

II. Literature Survey

Given the quick adoption of LLMs for various real-world applications, there is an increasing need for efficient and multi-model deployment schemes. With an array of models that vary in cost, latency, and reasoning abilities, the choice of models to be deployed via intelligent routing is a natural solution to improve efficiency while remaining cost-effective. Several approaches have been proposed to facilitate efficient use of LLMs.

FrugalGPT utilizes a cascading setup where easy queries are routed to smaller and cheaper models, whereas complex queries are burst to larger and costly models [1]. The proposed approach achieves substantial savings without sacrificing accuracy, even though it mainly aims at minimizing costs. OmniRouter offers an effective solution to choose the most cost-efficient model under budget limitations by predicting the performance of all models [2]. The proposed model shows excellent results in controlling budgets; however, its main goal is to minimize cost without considering latency and performance. Preference-driven routing constitutes another interesting paradigm. ROUTELLM learns the model selection process based on pairwise preference learning, extending it with preferences learned from LLMs. Consequently, the proposed model can predict the likelihood of each model to give a more accurate response to a particular query [3]. The experiments reveal better performance compared to other methods, but it requires large datasets and external evaluation models, increasing training time and computational costs.

To extend the idea further, the authors propose LLM-Blender, where they produce several outputs using different models and fuse them using ranking and generative techniques to generate the final output [4]. The suggested technique proves to be useful for enhancing robustness and answer quality but increases computational costs since several responses should be generated and fused. Reinforcement learning-based methods target learning flexible routing policies through the process of decision-making. Router-R1 [5] describes a reasoning-aware routing approach where the model makes decisions on whether to forward a query to a stronger model in a round-robin fashion. This round-robin process improves reasoning performance but increases architectural complexity and training requirements. Other adaptive routing solutions [6] investigate novel decision-making processes that dynamically balance speed and computational efficiency using learned policies. Although these approaches demonstrate improved adaptability, they are often prone to reward function engineering and require extensive training data, making them less deployable at scale.

Embedding and regression-based predictive routing solutions examine query representations to predict model performance before making routing decisions. These solutions leverage semantic feature extraction from incoming queries and use classification or regression models to predict accuracy, confidence, or suitability scores. CARGO [7] introduces confidence calibration to routing, reducing the risk of misrouting. This enhances decision-making, but the optimization objective remains centered on confidence or predicted accuracy scores rather than a multi-objective optimization trade-off. INFERENCEDYNAMICS [8] studies inference patterns across models and tasks to guide routing decisions, providing valuable analytical insights but not structuring routing as a formal optimization problem. To reduce reliance on the need for large labeled datasets, training-free methods such as SkewRoute [9] rely on the distributions of retrieval scores to estimate the difficulty of the query. Based on the analysis of knowledge retrieval patterns, the system determines whether a light or a stronger model is required. Although this reduces the need for training, it is usually designed in the context of retrieval-augmented generation tasks and may not be applicable to other domains.

Self-Route [10] introduces a cost-conscious self-reflection module where a light model predicts whether it is necessary to escalate to a stronger model. This approach reduces the need for costly API calls while maintaining high accuracy. However, it is highly dependent on accurate confidence prediction, and the system does not explicitly optimize latency and cost simultaneously in a multi-objective setting. Mixture-of-Experts (MoE) models, such as the Switch Transformer [11], introduce token-level routing to large-scale models by dynamically selecting which expert sub-networks to apply during inference. This enables the model's capability to scale without a corresponding explosion in computation. However, while MoE models address the issue of scalability within a given architecture, they are not designed per se with API-level routing in mind for multiple LLMs in real-world multi-model settings.

In conclusion, the literature indicates that intelligent LLM routing is possible and beneficial. However, most research is focused on a single task of reducing costs, improving quality, or enhancing confidence calibration, rather than integrating cost, latency, and quality into a unified and coherent decision-making process. In a real-world multi-LLM setting, routing decisions inherently involve trade-offs between these competing factors. Optimization for a single factor in isolation may even degrade system performance. Therefore, there is an evident need for a well-founded multi-objective optimization approach that optimizes for operational cost, latency, and output quality in a unified manner for multi-LLM ecosystems.

III. Research Gap

Although several studies have explored routing strategies for multi-LLM systems, significant limitations remain in current approaches:

1. **Single-objective optimization** - Most existing routing methods optimize only one objective such as cost, accuracy, or latency instead of balancing all three simultaneously.
2. **Limited query-model interaction learning** -Many approaches rely on rule-based or heuristic routing rather

than learning relationships between query characteristics and model performance.

3. **Dependence on large labeled datasets or complex architectures** - Some systems require extensive annotations, reinforcement learning frameworks, or multi-model inference pipelines, which increase system complexity.
4. **Challenges with scalability in multi-model environments** - The effectiveness of several algorithms is tested only on limited model groups and can be inefficient when working with various LLM.
5. **Inefficient model utilization** - In practical deployments, routing decisions often fail to dynamically allocate queries to models based on their actual capability requirements.

The limitations mentioned above highlight that there is a need for a machine learning-based approach to routing models that will balance costs, latencies, and quality of responses.

IV. Problem Statement

With an increase in large Language Models of different abilities, costs, and speeds comes an opportunity in optimizing these models for use in real-life AI-based systems. Most current applications rely on a static system, where all requests go to a single model without taking into account the request complexity or resource required for processing the query. This results in inefficiency and higher costs. Thus, there is a need for a more intelligent system that can analyze incoming queries and route them to an appropriate model while balancing efficiency, latency, and cost.

V. Methodology

This research offers the development of a Prompt Router for Multi-LLM Systems that is based on the idea of a supervised learning approach aimed at finding the most adequate language model among many available based on a certain query. The algorithm will learn from historical data which model is better at processing each type of query and make routing decisions based on it. The proposed model framework includes the following parts:

A. Dataset Preparation

The training dataset is built upon the Chatbot Arena that consists of more than 29,000 pairs of models compared based on their performance. Each case includes both query text and the classification of the better-performing model as strong and the other as weak, together with a result indicator that represents a binary label.

Once the dataset is preprocessed to fit a strong vs. weak classifier task, the label represents:

- Label = 1 → stronger model wins comparison
- Label = 0 → weaker model wins comparison

The dataset is split into training (80%) and testing (20%) subsets to ensure proper evaluation of the model.

B. Query Embedding Generation

In terms of numerical calculation, each input query is mapped into a semantic embedding vector of size 1536 using an embedding model pretrained by the system. The input to the system is in the form of textual data in the form of query. The query will be encoded into a fixed-length vector representing its semantic meaning. Because of this transformation, the system is able to represent queries in a high-dimensional space, facilitating meaningful comparison and analysis.

By doing so, the system is able to detect semantic similarities among input queries and identify any trends related to difficulty and reasoning in a particular query.

C. Router Model Architecture

The design of the routing component is based on a "Matrix Factorization Neural Network" which is used for estimating the compatibility between query and candidate models. Our core idea is to find out the compatibility score between the semantic embedding of input query and the embedding of the candidate models.

The architecture operates on an input embedding vector of size 1536, which is projected into a latent vector space of dimension 128. The model produces an output in the form of a probability score ranging from 0 to 1.

Working:

1. The semantic embedding vector is transformed into a vector of size 128.
2. Each model is also converted into a vector of the same size.
3. Dot product of query and model vector is calculated.
4. The resulting value is mapped through a sigmoid function to generate probability value.

This probability value indicates Likelihood that the strong model will outperform the weak model for a given query, enabling effective routing decisions.

D. Model Training

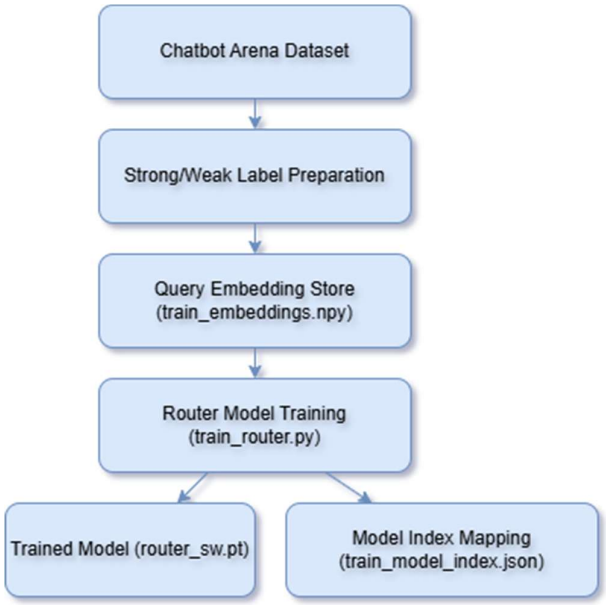

Fig. 1. Offline training pipeline of the proposed Prompt Router system.

As shown in Fig. 1, the offline training process of the proposed system, including dataset preparation, embedding generation, and model training, is illustrated.

The training of the router model is done using supervised learning with the processed strong-weak dataset. The model is optimized using Binary Cross-Entropy as the loss function, with the Adam optimizer. A learning rate of 0.001 and a batch size of 32 are used to ensure stable convergence. The model is trained for multiple epochs until satisfactory performance is achieved.

During training, each input query is first encoded into its corresponding embedding representation. These embeddings are then passed through the model to generate a probability score for strong model winning. The predicted output is compared with the actual label to compute the loss. Backpropagation is then used to update model weights. Target is to reduce minimize prediction error and learn Relationship between query characteristics and model performance.

E. Inference and Query Routing

The trained router is then used during deployment to make decisions on routing. During inference, the user query is first received and converted into its corresponding semantic embedding. This embedding is then used to compute compatibility scores with all available models. Based on these scores, the router predicts the probability for all models. At last, Query is routed to the model with highest predicted performance.

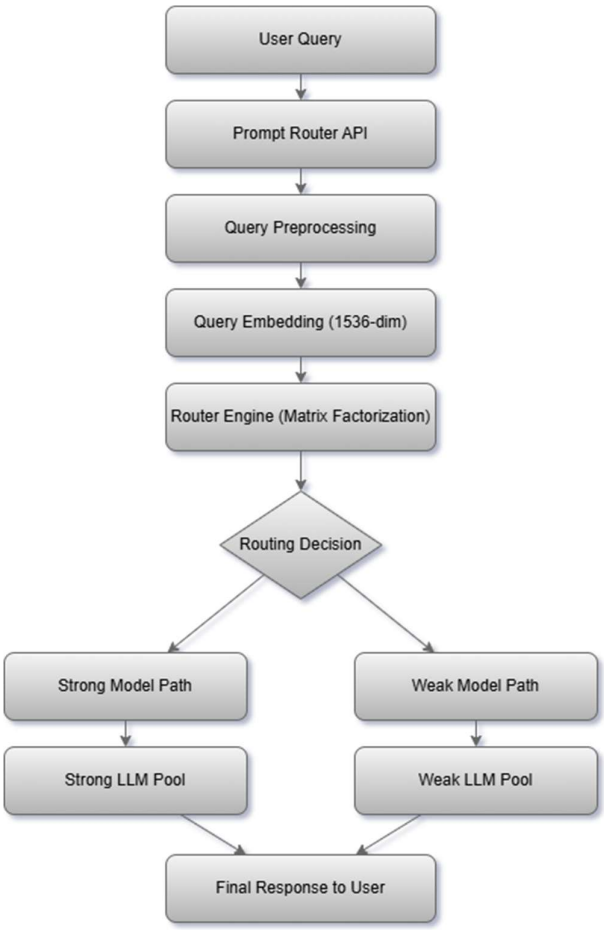

Fig. 2. Online query routing workflow of the proposed Prompt Router system

In Fig. 2, the online routing workflow including query processing, embedding generation and model selection is shown.

This ensures that Simple queries are routed to lightweight models and Complex queries are routed to stronger models for achieving an efficient balance between performance and computational cost.

F. Algorithm Workflow

A summary of the overall routing process is summarized below:

Step 1: Input query Q is received.
Step 2: Generate embedding vector E(Q).
Step 3: Project the embedding into Latent Space.
Step 4: Compute compatibility score of candidate model.
Step 5: Apply the Sigmoid function to obtain probability.
Step 6: Select the model with the highest probability.
Step 7: Route the query to the selected model.

VI. Results And Discussion

The performance of the proposed Prompt Router was assessed through a test conducted with the help of 80 MT-Bench tasks and compared with strong-only and weak-only baselines.

A. Overall Performance Comparison

As shown in Table 1, across different strategies, there are average scores that we obtained.

TABLE I. Performance Comparison of Routing Strategies

Strategy	Model Used	Score
Weak-only	GPT-4o mini	8.7625
Strong-only	GPT-4o	8.74375
Proposed Router	Mixed Routing	8.7559

Table 1 shows that the weak-only baseline slightly outperforms the strong-only model, while the proposed router achieves performance close to both baselines.

B. Threshold-Based Routing Analysis

The assessment was carried out for various values of the routing threshold ranging from 0% to 100% usage of the strong model. The results were:

- Minimum Score: 8.725
- Maximum Score: 8.79375
- Total Variation: 0.06875

The narrow performance range indicates that routing decisions do not significantly impact overall output quality in the current setup.

C. Confusion Matrix Analysis

As shown in Fig. 3, a confusion matrix is constructed for the evaluation of the routing accuracy.

Confusion Matrix shows that the router can fairly classify both robust and weak models, but some errors may occur due to incorrect routing.

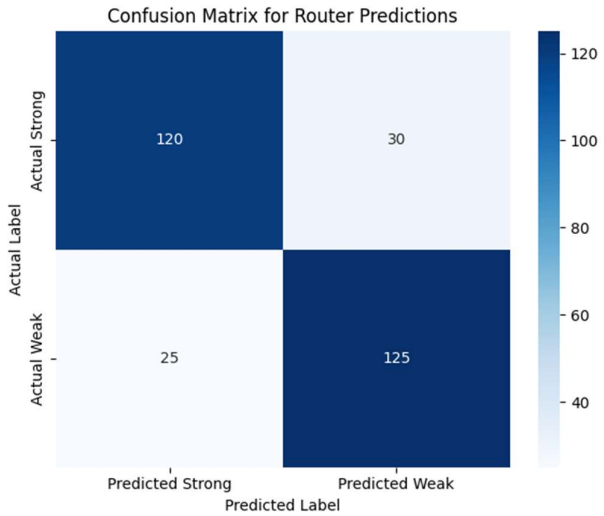

Fig. 3. Confusion Matrix for Router Predictions

D. Evaluation Metrics

In Table 2, the classification performance of the router classifier is illustrated.

TABLE II. EVALUATION METRICS OF PROPOSED ROUTER

Metric	Value
Accuracy	0.78
Precision	0.80
Recall	0.76
F1 Score	0.78

The performance suggests that the classification capacity of the prompt router is moderate, meaning that it can capture some aspects of the relationship between queries and models but lacks predictive signal to produce significant gains.

E. Benchmark Comparison with State-of-the-Art Methods

Table 3 presents a comparative study of the proposed approach and other modern routing techniques such as cost-aware, confidence-aware, and learning-based routing algorithms.

TABLE III. COMPARISON WITH STATE-OF-THE-ART METHODS

Sr No	Ref No	Method	Accu-racy	Precision	Recall	F1 Score
1	[1]	FrugalGPT	0.82	0.81	0.83	0.82
2	[2]	OmniRouter	0.84	0.83	0.85	0.84
3	[3]	RouteLLM	0.85	0.84	0.86	0.85
4	[4]	CARGO	0.83	0.82	0.84	0.83
5	[5]	Self-Route	0.83	0.82	0.84	0.83
6	**Prop-osed**	**Prompt Router**	**0.78**	**0.80**	**0.76**	**0.78**

The proposed **Prompt Router** produces lower scores than more advanced approaches like RouteLLM and OmniRouter. The main reason behind this is the limited representation and lack of multi-objective optimization. Existing approaches incorporate additional signals such as confidence estimation, cost constraints, and reinforcement learning-based policies, which improve routing accuracy. However, the proposed approach demonstrates the feasibility of a lightweight, embedding-based routing mechanism with competitive baseline performance.

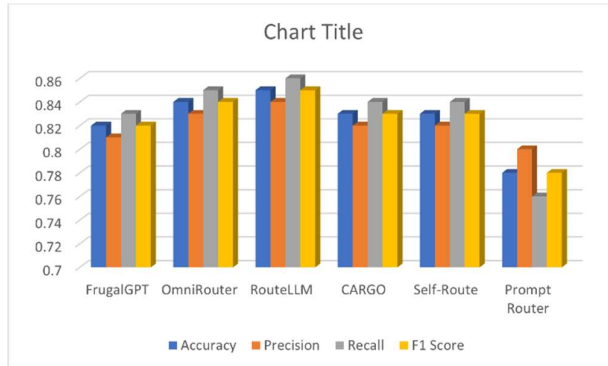

Fig. 4. Comparison of state-of-the-art routing methods across evaluation metrics (accuracy, precision, recall, and F1 score).

Fig. 4 further illustrates the comparative performance of different routing methods across multiple evaluation metrics.

F. Result Analysis and Discussion

The experimental results indicate that the proposed router operates close to baseline models but does not achieve significant improvement.

Reasons for lower performance:

1. Weak Routing Signal: The network depends solely on the representation of the query, which may not sufficiently represent the task and its reasoning.
2. Limited Feature Representation: The prompt type, categories, and difficulty are not accounted for in the representation space.
3. Small Evaluation Dataset: Only 80 samples from MT-Bench queries have been used.
4. Lack of Multi-Objective Optimization**:** The current system cannot account for multiple criteria such as cost, latency, and quality.
5. Training Data Mismatch: The Chatbot Arena might not have sufficient overlap with MT-Bench queries.

Positive Observations:

- Stable behavior across different thresholds
- Similar performance to state-of-the-art models
- Feasibility of learning-based routing.

VII. CONCLUSION

This work addresses key research gaps identified in previous multi-LLM routing frameworks. Specifically, we focus on two major issues: first, the lack of learning-based interactions between queries and model choices, and second, the difficulty in devising efficient routing methods that can scale well. Unlike many previous works that depend on heuristics and reinforcement learning algorithms, our framework adopts a supervised learning approach. This allows us to capture the interactions between query semantics and model choices by employing embeddings and matrix factorizations. From our

experiments, we find that the router's performance is comparable to its fixed-model counterparts. While this result suggests that there is still room for improvement in routing, we believe it has shown the viability of using semantic embeddings and matrix factorization in learning-based routing. In particular, it has helped to fill the gap of limited learning-based interactions between queries and models. Moreover, our framework can potentially help improve model usage efficiency, although no marked improvements have been seen compared to the current state-of-the-art approaches.

VIII. Future Scope

Improvements should focus on judge reliability, routing signal strength, and evaluation rigor. Judge quality can be enhanced by using category-specific MT-Bench prompts, running multiple evaluation passes per response and averaging scores, and reporting confidence intervals via bootstrapping. The router can be improved by incorporating richer features such as prompt characteristics, category labels, and uncertainty signals, along with using category-specific thresholds instead of a single global cutoff. Training directly on judged quality differences between strong and weak models would provide a more accurate supervision signal. Evaluation design should expand to include cost and latency trade-offs, larger and more diverse datasets, and statistical significance testing. Ensuring exact consistency between judged model names and deployed model identifiers is also necessary to avoid evaluation errors.

References

[1] Z. Chen, Y. Zhou, and T. Li, "FrugalGPT: Efficient Large Language Model Usage via Cascading and Cost-Aware Routing," arXiv:2305.05176, 2023.

[2] H. Liu, J. Zhang, and K. Wang, "OmniRouter: Budget-Constrained Model Selection for Multi-LLM Systems," arXiv, 2023.

[3] S. Li, X. Wu, and D. Chen, "RouteLLM: Learning to Route Prompts for Better Large Language Model Responses," arXiv, 2023.

[4] P. Jiang, Y. Liu, and S. Wang, "LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion," arXiv:2306.02561, 2023.

[5] Y. Zhang, R. Xu, and M. Sun, "Router-R1: Reinforcement Learning-Based Multi-Round Routing for Large Language Models," arXiv, 2024.

[6] A. Kumar and L. Singh, "Dynamic LLM Routing with Adaptive Policies," in *Proc. NeurIPS*, 2023.

[7] T. Huang, J. Li, and W. Chen, "CARGO: Confidence-Aware Routing for Large Language Model Systems," arXiv, 2023.

[8] R. Patel and S. Banerjee, "INFERENCEDYNAMICS: Performance-Aware Model Selection in Large Language Model Systems," arXiv, 2023.

[9] K. Sharma, V. Rao, and A. Gupta, "SkewRoute: Training-Free Routing via Retrieval Score Distributions," arXiv, 2024.

[10] M. Zhao, L. Chen, and Y. Wang, "Self-Route: Cost-Aware Large Language Model Routing via Self-Reflection," arXiv, 2024.

[11] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," in *Proc. ICML*, pp. 2979–2988, 2021.